



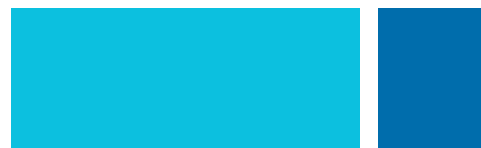
SINGLETON E ISP

Pedro Webber



INTERFACE SEGREGATION PRINCIPLE

- INTERFACES PEQUENAS, ESPECÍFICAS E COESAS
- CADA CLASSE DEVE IMPLEMENTAR APENAS O QUE REALMENTE PRECISA
- “NENHUM CLIENTE DEVE SER FORÇADO A DEPENDER DE MÉTODOS QUE NÃO UTILIZA.”



PROBLEMAS CAUSADOS POR INTERFACES GRANDES

- Classes obrigadas a implementar métodos irrelevantes.
- Maior acoplamento entre componentes.
- Mudanças em uma interface quebram várias classes.
- Dificulta manutenção e testes.



COMO O ISP SOLUCIONA ISSO?

- DIVIDE GRANDES INTERFACES EM VÁRIAS MENORES.
- ORGANIZA RESPONSABILIDADES COM PRECISÃO.
- CADA CLIENTE IMPLEMENTA APENAS A INTERFACE NECESSÁRIA.
- AUMENTA A FLEXIBILIDADE E A ESTABILIDADE DO SISTEMA. CLIENTE IMPLEMENTA APENAS A INTERFACE NECESSÁRIA.





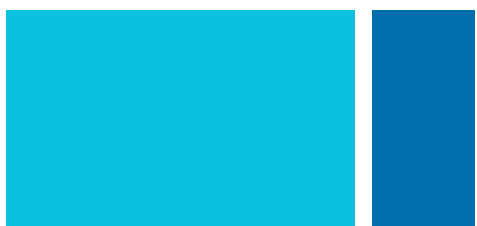
BENEFÍCIOS DO ISP

BAIXO ACOPLAMENTO E ALTA COESÃO.

CLASSES MAIS SIMPLES E FOCADAS.

MENOR IMPACTO DE MUDANÇAS

FACILITA TESTES UNITÁRIOS E
REUTILIZAÇÃO DE CÓDIGO.



EXEMPLOS

SEM ISP:

```
class IAuthService {  
public:  
    virtual void login(const string& user, const string& pass) = 0;  
    virtual void registerUser(const string& user, const string& pass) = 0;  
    virtual void resetPassword(const string& email) = 0;  
};
```

Um admin que só precisa logar e resetar a senha é obrigado a implementar registeruser()

EXEMPLOS

COM ISP:

```
class ILogin {
public:
    virtual void login(const string& user, const string& pass) = 0;
    virtual ~ILogin() = default;
};

class IRegister {
public:
    virtual void registerUser(const string& user, const string& pass) = 0;
    virtual ~IRegister() = default;
};

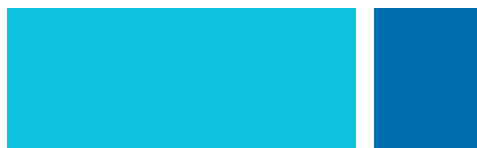
class IForgotPassword {
public:
    virtual void resetPassword(const string& email) = 0;
    virtual ~IForgotPassword() = default;
};
```

Interfaces segregadas

SINGLETON

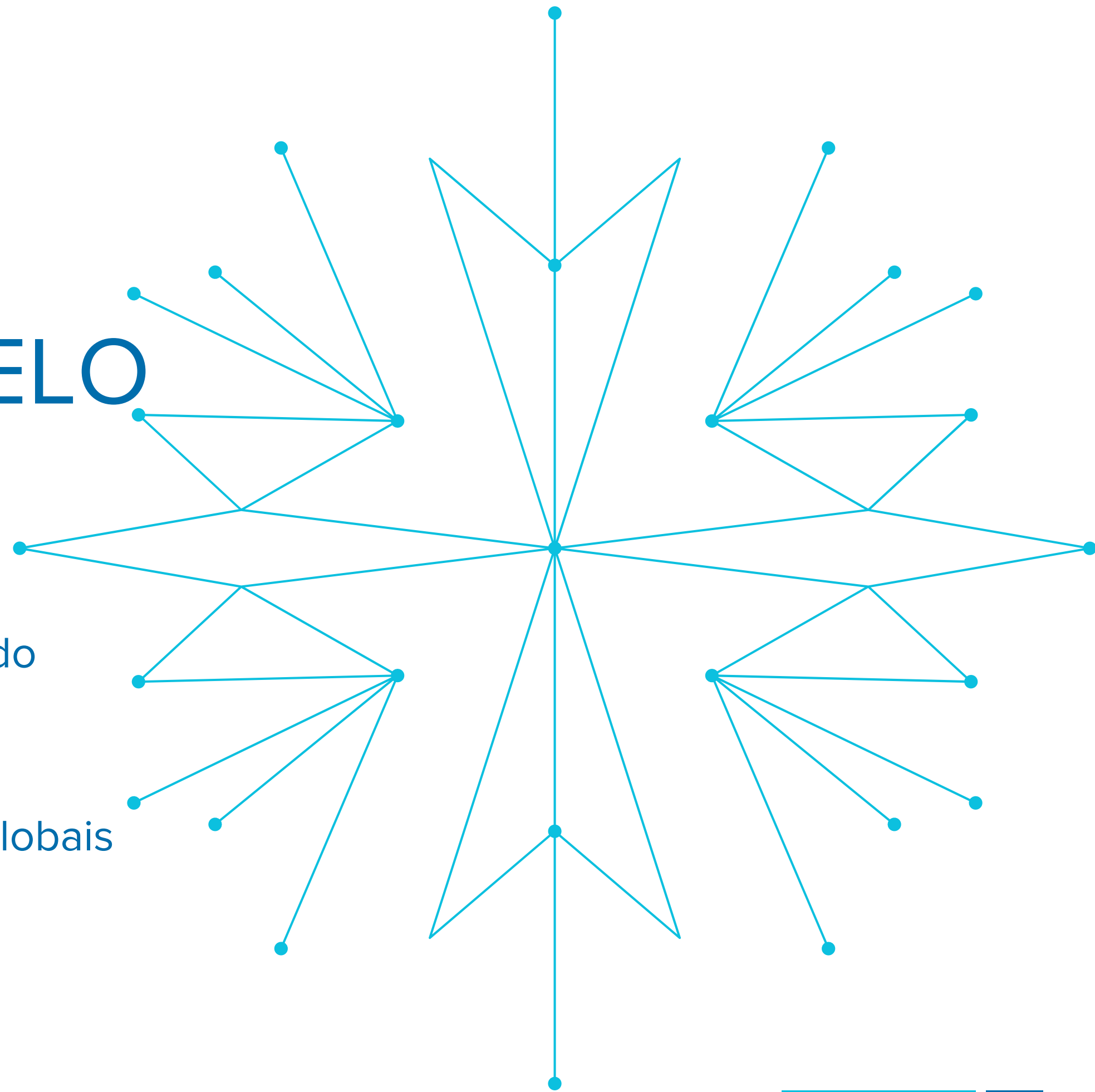
“Singleton é um padrão que garante que uma classe tenha apenas uma instância, e fornece um ponto global de acesso a ela.”

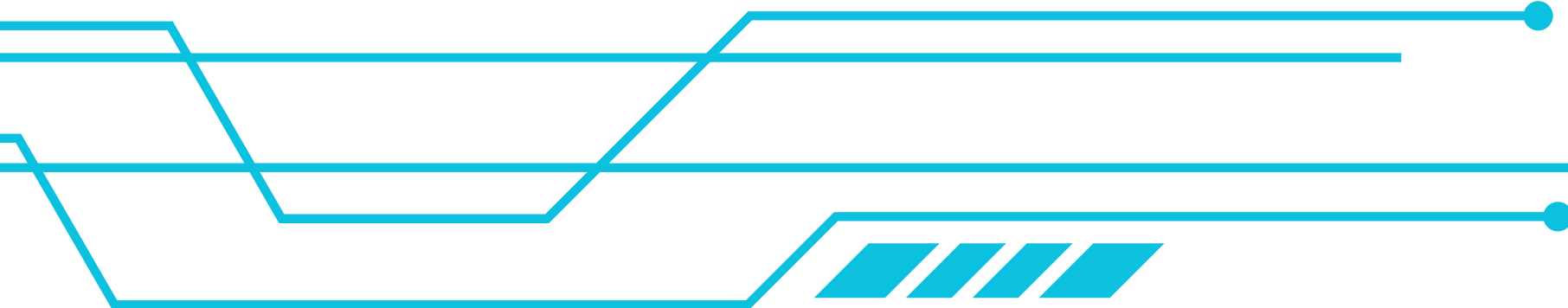
- Controlar o acesso a um recurso compartilhado
- Evitar múltiplas instâncias desnecessárias ou perigosas
- Centralizar responsabilidades como logging, configurações, cache, conexão com banco de dados, etc.



PROBLEMAS SOLUCIONADOS PELO SINGLETON

- Múltiplas instâncias de uma classe gerando conflitos
- Controle fragmentado de recursos
- Falta de consistência em configurações globais



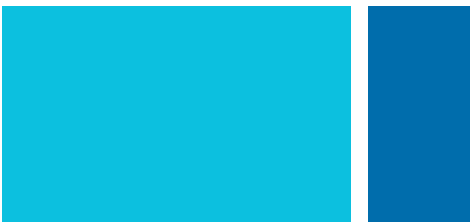


ESTRUTURA SINGLETON

CONSTRUTOR PRIVADO

INSTÂNCIA ESTÁTICA PRIVADA DA PRÓPRIA CLASSE

MÉTODO PÚBLICO ESTÁTICO QUE RETORNA A INSTÂNCIA (LAZY INSTANTIATION)



EXEMPLO

```
class Logger {
private:
    Logger() {} // Construtor privado
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;

public:
    static Logger& getInstance() { //instancia estatica privada
        static Logger instance;
        return instance; //lazy instantiation
    }

    void log(const string& message) {
        cout << "[LOG]: " << message << endl;
    }
};
```

```
int main() {
    Logger::getInstance().log("Usuário logado.");
    Logger::getInstance().log("Erro de autenticação.");
    return 0;
}
```

